Reflections on Trusting /proc

Comparing Process Identity, Executable Files, and Mapped Memory on Linux

Ivan Gaydardzhiev
Independent Research
7 November 2026

The files under `/proc` are witnesses, and they are not equally independent. Some are produced by the kernel from its own bookkeeping, and some are bytes in the writable memory of the process, passed through by the kernel. I reproduced on one Linux 7.1.5 machine what `ps`, `top` and `netstat` read, what a process can change in its own account, and what the kernel allowed and refused. I built `wtn`, a C program which compares the executable, the file its path names and the mapped memory. Its testbed of 19 programs passed 19 of 19 as root, and this paper states what each disagreement can and cannot mean.

1. Introduction

Here is one process, as four sources report it.

```
$ ps -o pid,comm,args -p 19095
    PID COMMAND         COMMAND
  19095 03-argv_overwri [kworker/0:1]
$ cat /proc/19095/comm
03-argv_overwri
$ tr '\0' ' ' </proc/19095/cmdline; echo
[kworker/0:1]
$ readlink /proc/19095/exe | sed 's,.*/testbed,.../testbed,'
.../testbed/bin/03-argv_overwrite
```

`ps` shows a kernel thread. `comm` shows a truncated program name. `cmdline` is where `ps` found its answer. `exe` names a test program of mine. The question of this paper is which of these is right when they disagree, and what right means.

Processes became files in the Eighth Edition research system, in the work Killian described in 1984 [1]. Plan 9 kept the idea: in its `proc(3)`, `text` stands for the file the process was executed from and `mem` is its memory image [2]. The BSD systems went another way. The FreeBSD `procfs(5)` page marks the file system deprecated and sends the reader to `libprocstat(3)` and `kvm(3)` [3], and OpenBSD removed procfs in release 5.7 in 2015 [4]. On Linux the common tools still read `/proc`, as Section 2 shows.

The starting point is a talk by Kalvāns at SREcon26 EMEA [5].

`/proc` is a set of witnesses, not the kernel's absolute truth, and one distinction explains nearly every case below. Some files are produced by the kernel from its own bookkeeping: `exe`, `maps`, `status`, the fields of `stat`, the contents of `mem` as mapped. Others are bytes in the process's own writable memory that the kernel passes through: `cmdline` and `environ`. A tool that trusts the second kind is trusting the process's statement about itself. The tool `wtn` compares witnesses whose producers differ, so that agreement means something and disagreement is informative. A disagreement is a fact about the system. Many are innocent: a multi-call binary, a symlinked interpreter, a container, a binary replaced by a package upgrade.

Everything here ran on one machine: Kali, Linux 7.1.5, gcc 16.2.0, procps-ng 4.0.7, net-tools 2.10, man-pages 6.19. The testbed ran as real root. The other captures ran as uid 1000.

2. Method

Run a tool under `strace -e trace=file` and find the `/proc` names it opens. Read the manual page for each name [6]. Reproduce the behaviour on a testbed process and keep the output. This is the paper's only technique. Here is the end of the trace of `ps -o pid,comm,args -p 1`:

```
openat(AT_FDCWD, "/proc/1", O_RDONLY|O_DIRECTORY) = 3
openat(3, "status", O_RDONLY)           = 4
openat(3, "stat", O_RDONLY)             = 4
openat(3, "status", O_RDONLY)           = 4
openat(3, "cmdline", O_RDONLY)          = 4
```

`ps` opens the directory and then opens names relative to it, so a search for the string `/proc/1/stat` finds nothing. `top -b -n1` also opens relative names, `stat` and `statm`, and nothing else per process. `netstat` builds full paths. `netstat -tlnp` as uid 1000 gave this:

```
# netstat -tlnp as uid 1000
calls 2166 denied 241
relative opens of stat or comm: 0
   1496 "/proc/N/fd/M"
    374 "/proc/N/attr/current"
    214 "/proc/N/fd"
     60 "/proc/N/cmdline"
      2 "/proc"
      1 "/proc/N/mountinfo"
      1 "/proc/net/tcp6"
      1 "/proc/net/tcp"
      1 "/proc/filesystems"
```

All of these tools read text the kernel formats. They differ in which file they stop at.

3. Who writes each file

The second column is the distinction of Section 1. A third class appears in it: a value the kernel stores in its own memory but the process may set.

file	produced by	what it holds	what trusting it means
`comm`	kernel, set by process	thread name	accepting a name the thread chose
`cmdline`	process memory	bytes between `arg_start` and `arg_end`	accepting the process's statement
`environ`	process memory	bytes between `env_start` and `env_end`	the same, with the range itself movable
`exe`	kernel	link to the executed file	accepting the file, not what is mapped now
`maps`	kernel	mappings, permissions, backing names	accepting layout, not contents
`map_files`	kernel	one link per file mapping	reaching the backing file; opening needs root
`mem`	kernel, pages as mapped	the bytes present now	seeing the current code, not who wrote it
`stat`	kernel	`comm` in parentheses, then numbers	parsing past a name the process chose
`status` Uid	kernel	real, effective, saved, filesystem ids	identity only, not privilege
`status` CapEff	kernel	effective capability mask	privilege held now
`status` NSpid	kernel	pid in each pid namespace	locating the process in nested namespaces
`status` PPid	kernel	current parent	the present parent, not the origin
`root`, `cwd`	kernel	links; `root` gives the process's view of files	seeing files as the process sees them
`fd/`	kernel	one link per descriptor	ownership or root needed to look
`net/*`	kernel	tables of the reader's network namespace	sockets, with no process named
`mountinfo`	kernel	mounts in the process's namespace	the mount layout behind a path
`cgroup`	kernel	control group membership	placement, which survives a parent's exit

`proc_pid_cmdline(5)` says that changes a process makes to its argv strings show up in the file, and advises thinking of it as the command line the process wants you to see. Case 03 proves it in Section 1, with no privilege. `environ` is documented as the initial environment, yet case 08 wiped a variable from those bytes in place, and the file changed. The kernel keeps the range and the process owns the bytes. `prctl(PR_SET_NAME)` is the third class: `comm` lives in the kernel, and the thread writes it with one call that needs no capability, as case 04 shows.

4. The tools and the kernel disagree

`comm` truncation. Case 01 runs a program named `systemd-resolved-long-name` and `comm` holds `systemd-resolve`, 15 bytes. Setting 26 bytes with `PR_SET_NAME` left 15. `TASK_COMM_LEN` is 16 with the NUL, per `proc_pid_comm(5)`. Kernel threads exceed it. Pid 3 here is `pool_workqueue_release`, 22 bytes, and `comm`, `stat` and `status` agree on the name:

```
31 31 rcu_exp_par_gp_kthread_worker/0 stat=rcu_exp_par_gp_kthread_worker/0
```

```
pid 3: comm=pool_workqueue_release stat=(pool_workqueue_release) status=pool_workqueue_release
```

The limit of 15 applies to names of user processes. I found no limit for kernel threads: the longest in this capture is 31 bytes. `proc_pid_stat(5)` says the field is truncated at 16 and does not mention the exception.

The `stat` parsing ambiguity. Field 2 is `comm` in parentheses, and `comm` may hold spaces and parentheses. I set it to `a) R 1 1 (b`:

```
$ cut -d' ' -f1-8 /proc/19996/stat
19996 (a) R 1 1 (b) S 19057
$ awk '{print "ppid by field 4:", $4}' /proc/19996/stat
ppid by field 4: 1
$ grep PPid /proc/19996/status
PPid:   19057
```

Splitting on spaces yields a parent of 1. `ps` was not misled. `wtn` searches for the last `)`, which the name cannot move past.

`ps` and kernel threads. `ps` reads `cmdline`, and when it is empty prints the name in brackets, as `ps(1)` says [7]. A kernel thread has an empty `cmdline`, and so can a user process. `PR_SET_MM_MAP` let a process at uid 1000 empty its own command line:

```
$ wc -c </proc/20003/cmdline
0
$ ps -o pid,comm,args -p 20003,2
    PID COMMAND         COMMAND
  20003 kworker/0:9     [kworker/0:9]
      2 kthreadd        [kthreadd]
$ grep -H Kthread /proc/20003/status /proc/2/status
/proc/20003/status:Kthread:     0
/proc/2/status:Kthread: 1
```

The brackets follow an empty `cmdline`, which the process controls. The line `Kthread:` in `status` is the kernel's answer. It is not in `proc_pid_status(5)`. `wtn` stops on a missing `VmSize:` line, which is also kernel bookkeeping.

`fd/` and sockets. `/proc/N/fd` needs ownership or root, so `netstat` as uid 1000 warns that it cannot name processes it does not own. Those are the 241 denials in the trace above. The link between a socket and a process is one inode number, documented in `proc_pid_fd(5)`:

```
$ ls -l /proc/20015/fd/3 | sed 's/.* -> //'
socket:[163825]
$ awk '$10 == 163825 {print $2, $4, "uid=" $8, "inode=" $10}' /proc/net/tcp
0100007F:B813 0A uid=1000 inode=163825
```

`/proc/net/tcp` names a uid and an inode, not a process. To name the process a tool must walk every `fd/` directory, which explains the 1496 lookups above. The name printed afterwards comes from another file. net-tools 2.10 opened `cmdline`, 60 times, and never `comm` or `stat`. With `argv[0]` set to `zzz`:

```
$ cat /proc/20027/comm
p-sock
$ tr '\0' ' ' </proc/20027/cmdline; echo
zzz
$ netstat -tlnp 2>/dev/null | grep 47123
tcp        0      0 127.0.0.1:47123         0.0.0.0:*               LISTEN      20027/zzz
```

So `netstat` reports the process's statement, and `top`, which opens `stat`, reports the name there.

5. A process edits its own testimony

Each case is a program in `testbed/`. Listings are in the appendix.

`argv[0]` is a convention. Case 02 executes `sleep` with `argv[0]` set to `camouflage`. `execve` takes what the caller gives, and `wtn` reports `ID_ARGV_EXE`. Case 03 needs no cooperating parent: it writes over its own argv after start. Case 04 sets `comm` to `septicon`. None of the three needed a capability, and all three are visible only by comparison with `exe`, the kernel's witness.

Loader variables. In case 07, `LD_PRELOAD=./libp.so` is in `environ` and the library is in `maps`, and the two witnesses agree. In case 08 the library's constructor wipes the variable from the environment bytes. `environ` falls silent. `maps` still shows `libs.so`, because the kernel produced that line and the process only

wrote its own memory. Case 09 repeats case 07 with `LD_LIBRARY_PATH`, and `ld.so(8)` documents both variables.

Environment replacement. Case 10 copies its environment to the heap without the `LD_` variables and points the kernel at the copy with `PR_SET_MM_ENV_START` and `PR_SET_MM_ENV_END`. Now `/proc/N/environ` reads clean. The old strings remain:

```
11108 ! ENV_RELOCATED env=0x55b986dcc010-0x55b986dcc9e5 stack=0x7fff90297000-0x7fff902b8000
11108 ? ENV_STRAY_LOADER_VAR name=LD_PRELOAD addr=0x7fff902b7fd4
```

The first line compares the kernel's range with the kernel's `[stack]`. The second reads the stack through `mem`. The original strings stay in memory after a swap on this kernel. The privileges were not what the manual says:

```
# uid 1000, no capability
PR_SET_MM_ENV_START: Operation not permitted (errno 1)
PR_SET_MM_EXE_FILE: Operation not permitted (errno 1)
PR_SET_MM_MAP exe_fd, exe mapped: Operation not permitted (errno 1)
PR_SET_MM_MAP new environment: ok
# uid 0 in a user namespace
PR_SET_MM_ENV_START: Operation not permitted (errno 1)
PR_SET_MM_EXE_FILE: Operation not permitted (errno 1)
PR_SET_MM_MAP exe_fd, exe mapped: Device or resource busy (errno 16)
PR_SET_MM_MAP new environment: ok
```

`PR_SET_MM(2const)` says the caller must have `CAP_SYS_RESOURCE`. That held for the direct calls, which refused uid 1000 and uid 0 in a user namespace with `EPERM`, and which worked for real root in cases 10 and 11. `PR_SET_MM_MAP` ran with no capability, on a kernel built with `CONFIG_CHECKPOINT_RESTORE`. `PR_SET_MM_ENV_START(2const)` says the address should belong to the stack, and case 10 gave it a heap address.

Repointing `exe`. `PR_SET_MM_EXE_FILE(2const)` says all executable mappings of the old file must go first, and lists `EBUSY` for a second attempt. Case 11 as first written made the call on `/bin/true` directly. The kernel returned `EBUSY` (errno 16) and the case was skipped in the first two root runs. That output was not kept. The user namespace line above shows the same errno through the map call. The case now moves each of its mappings to anonymous memory with `mremap`, repoints the link, maps a page of `/bin/true` and flips one byte. As real root:

```
11129 ? ID_COMM_EXE comm=11-exe_repoint exe=/usr/bin/true
11129 ! MEM_MISMATCH map=0x7f56540c0000-0x7f56540c1000 off=0x1000 path=/usr/bin/true via=exe d ...
11129 ? ANON_EXEC map=0x401000-0x496000 perms=r-xp name=-
```

So `exe` is kernel bookkeeping that real root edited here, and only after giving up every file behind its code. EXE and both PATH witnesses then agree on `/usr/bin/true`. The running code appears as `ANON_EXEC`, which needs no flipped byte.

6. The kernel is consistent but relative

Nobody edited the files below. Their answers depend on the viewer or the moment.

Deleted binaries. Case 05 unlinks its binary after `exec`. The link gains `(deleted)`, and `proc_pid_exe(5)` documents that. Opening the link still gives the file:

```
11006 . EXE path=/var/tmp/wt.jT5R7I/05-deleted/sleep\040(deleted) comm=sleep argv0=sleep
11006 ? EXE_DELETED path=/var/tmp/wt.jT5R7I/05-deleted/sleep\040(deleted)
11006 . HASH witness=exe sha=83c0db85657e2cd1799b5e113317252d7865dd78157148d8757fab0ede52b1db  ...
```

The hash equals the one in case 00, the plain copy. `readlink` gives a name, and opening gives the file. I also replaced a running binary with `mv`, as a package upgrade does: `wtn` printed the same `EXE_DELETED` finding, and nothing has happened but the upgrade. Case 06 reaches the same state with `memfd_create(2)`. The link reads `/memfd:ghost (deleted)`, a file that never had a directory entry, and the bytes hash to the same value again. The string `(deleted)` alone is ambiguous, as `proc_pid_maps(5)` admits for its own use of it.

`readlink` against a root. The text of `exe` depends on who reads it. I ran a static program in a chroot, in a user namespace, printing its own link, then read the link from outside:

```
inside pid=1 exe=/bin/p
outside pid=20065 exe=$B/R/bin/p
outside root=$B/R
20065 ? PATH_ROOT_MISSING path=$B/R/bin/p
```

Inside it reads `/bin/p` and outside it reads the full path. So a chroot does not mislead `sha256sum $(readlink /proc/N/exe)` on the host: the original case 16 expected it would, and failed in both root runs that used it. The path is correct for the reader. The witness ROOT, which appends that path to `/proc/N/root`, finds nothing, and that is the finding above.

A mount namespace is different, and it is the container case. Case 16 binds another program over `/bin/sleep` in a private mount namespace.

```
11240 . EXE path=/usr/bin/sleep comm=sleep argv0=/bin/sleep
11240 . HASH witness=exe sha=54ac9dca0576f9104931709ad4441066dc23135a8c2470b864630eeb3f01f233  ...
11240 . HASH witness=root sha=54ac9dca0576f9104931709ad4441066dc23135a8c2470b864630eeb3f01f233 ...
11240 . HASH witness=host sha=83c0db85657e2cd1799b5e113317252d7865dd78157148d8757fab0ede52b1db ...
11240 ? PATH_HOST_MISMATCH path=/usr/bin/sleep exe_sha=54ac9dca0576f9104931709ad4441066dc23135 ...
```

EXE and ROOT agree and HOST differs. Nothing here is false: the link is right inside the namespace and the bare path is right outside it. `/proc/N/root` gives the process's view, as `proc_pid_root(5)` describes. `mountinfo` records a bind, by its manual page. I did not capture it for this case. Case 18 is the mirror: the bind arrives after `exec`, so EXE and HOST agree and ROOT differs, which `wtn` reports as `PATH_ROOT_MISMATCH`.

Capabilities against uid. Case 14 runs as uid 65534 with `CapEff` 4, bit 2, `CAP_DAC_READ_SEARCH`, granted by a file capability. Uid and `CapEff` are separate kernel fields. Case 15 is a setuid copy of `sleep`: `IDS_SPLIT ruid=65534 euid=0`. A tool printing one uid has chosen one of the four in `status`.

Lineage. Case 17 forks twice and the middle process exits. `PPid` is then 1, and that is correct: the kernel reports the present, not the origin. What remains is in other fields:

```
11265 . LINEAGE ppid=1 cgroup=/user.slice/user-1000.slice/session-2.scope tty=34818
```

The cgroup and the tty number are evidence of where the process began.

7. Which witnesses are independent of which

Figure 1 shows the chain. A tool reads text, the kernel formats the text from its structures, and those structures point at process memory and at backing files. Two witnesses are independent only as far as their paths through this chain differ, and only where one producer cannot write what the other reports.

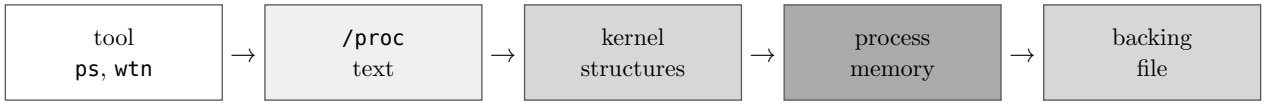


Figure 1: The chain of witnesses. The darker box is writable by the process under examination. `cmdline` and `environ` pass through to it. `exe`, `maps` and `status` stop at the kernel structures. `mem` reaches it, and `exe` opened as a file reaches the last box.

witnesses	a disagreement can mean	it cannot mean
`comm`, `cmdline`, `exe`	a rename, a wrapper, an interpreter, a multi-call binary	that the code differs: none of these reads code
`exe` bytes, ROOT path	the path was rebound or replaced after `exec`	anything about memory
ROOT path, HOST path	the two views of files differ, as in a container	that either is wrong
`exe` bytes, `mem`	text changed after mapping: a patch, a breakpoint	who changed it, or why
`environ`, stack contents	the range moved or the bytes were rewritten	what the environment was
`maps`, `environ`	a library is present that the environment does not explain	how it arrived
Uid, `CapEff`	privilege came by another route, such as a file capability	that it was used
`PPid`, cgroup, tty	the parent has gone	who the parent was

`comm` and `cmdline` agreeing means little, since one process can set both. `exe` and `mem` agreeing means more, since moving `exe` took a capability and the loss of every file mapping, as Section 5 shows. All of them still pass through one kernel, and that is the subject of Section 9.

The question of Section 1 has an answer. Each source is right about its own producer: `ps` about what the process says, `comm` about the thread's last name, `exe` about the file the kernel recorded, `mem` about the bytes present. A tool should say whose testimony it repeats.

8. wtn

8.1. Invocation

`wtn [-a] [-x] [-v] [-T] [pid ...]` takes decimal pids, at most 4096, and ignores repeats. `-a` examines the named pids, then every numeric entry of `/proc` except `wtn` itself and the pids already done. After that walk it stats `/etc/ld.so.preload`, the one check tied to no pid. `-x` adds a hex dump to the three findings that compare bytes. `-v` prints the `.` lines. `-T` hashes three FIPS 180-4 test vectors, the empty string, `abc` and a million `a`, prints `ok` or `FAIL` and exits with 0 or 2. A bad option, a bad pid, or no pid without `-a` prints the usage line and exits with 2. Every file is opened read-only. The tool repairs nothing.

8.2. The three witnesses

EXE is the bytes of the file opened through `/proc/N/exe`, hashed whole with SHA-256. The kernel keeps a record of that file. The open reached it after its name was deleted in case 05, and for a memfd in case 06.

PATH is the file named by `readlink` of the same link, with any `(deleted)` suffix removed, opened two ways. ROOT opens `/proc/N/root` followed by the path, which resolves the name in the process's view of the file system. HOST opens the bare path in the view of `wtn`, which is what `sha256sum $(readlink /proc/N/exe)` does. Both opens use `O_NONBLOCK` and accept only regular files, because the target controls the path and a fifo there would otherwise hang the tool.

MEM is the bytes of each `r-x` file-backed mapping, read from `/proc/N/mem` and compared with the same range of the backing file.

The producers differ. EXE answers from the kernel's record of the process. PATH answers from the file system's name lookup at this moment. MEM answers from the pages present. A file replaced on disk separates PATH from EXE. A private mount separates HOST from ROOT. A patched page separates MEM from EXE.

8.3. The steps

Each pid passes through steps 0 to 6 in order. If a read of a `/proc/N` file fails with `ENOENT` or `ESRCH`, the process has gone: `wtn` prints `GONE` and ends the pid. Any other failure prints `UNVERIFIED`, naming the file, and the work continues where it can.

Step 0 reads `status`. With no `VmSize:` line the process has no user address space, and `wtn` prints `KTHREAD` and ends the pid. A zombie has no such line either and takes the same path. A failed read here also ends the pid.

Step 1 reads `comm`, the `exe` link and `cmdline`. `argv[0]` is the bytes up to the first NUL. It emits `ID_COMM_EXE`, `ID_COMM_ARGV` and `ID_ARGV_EXE`, then the information line `EXE`.

Step 2 examines the link text. A suffix `(deleted)` emits `EXE_DELETED`. A prefix `/memfd:` emits `EXE_MEMFD`. A memfd link carries both, so a memfd reports both. It then opens `exe`, keeps the descriptor for later steps, and hashes the file in 64 KiB reads. The information line is `HASH witness=exe`. If the open or the hash fails, step 3 is skipped.

Step 3 hashes ROOT and HOST and compares each hash with EXE. For a deleted file or a memfd no path exists to open, so it prints `PATH_GONE` and stops. If ROOT differs it emits `! PATH_ROOT_MISMATCH`, because the process's own view contradicts the file it runs. `ENOENT` or `ENOTDIR` on ROOT can also mean the process died, so `wtn` tests `/proc/N/stat`: `GONE` if that has vanished, otherwise `PATH_ROOT_MISSING`. If HOST differs it emits `? PATH_HOST_MISMATCH`, because a container is an innocent cause. An absent HOST is the information line `PATH_HOST_MISSING`. Any other failure, including a path that is not a regular file, prints `UNVERIFIED which=root` or `which=host`.

Step 4 parses `maps`, compares memory as described below, and then walks the table four times, emitting `ANON_EXEC`, `WX_MAP`, `LIB_DELETED` and `LIB_ODD_PATH`. It ends with the line `MAPS count=N exec_checked=M`, where M counts the mappings actually compared. A count of 0 on a live process means nothing was checked.

Step 5 parses `stat`, tests the argv and environ ranges, reads `environ`, and scans `[stack]` and `[heap]`. It can emit `ENV_RELOCATED`, `ARGV_RELOCATED`, `ENV_LOADER_VAR` and `ENV_STRAY_LOADER_VAR`.

Step 6 reads `status` again and emits `CAPS_UNPRIVILEGED`, `IDS_SPLIT`, `NS_NESTED` and `LINEAGE`.

8.4. The memory comparison

`maps` is read in 64 KiB blocks and split into lines of at most 4095 bytes. A longer line prints `UNVERIFIED which=maps_line` once, and a line that does not parse prints `which=maps`. Each line gives start, end, permissions, offset and path, after skipping device and inode. The escape `\012` in a path is decoded back to a newline, because `readlink` returns the raw byte and the two strings are compared. The table holds 4096 mappings. The next one prints `which=maps_cap`, and later mappings are counted and not stored. `[stack]` and `[heap]` are kept apart from the table, so step 5 has them in any case.

A mapping is compared if its permissions begin `r-x` and its path begins `/`. A path beginning `/memfd:` is compared only if it is the main executable. The main executable is the mapping whose path equals the link text exactly, suffix included.

The backing file is chosen in order. The main executable uses the descriptor from step 2, reported as `via=exe`, so a deleted file and a memfd are compared with their real bytes. Any other mapping opens `/proc/N/map_files/START-END`, reported as `via=map_files`. If that fails and the path has no `(deleted)` suffix, it opens the path through `/proc/N/root`, reported as `via=root_path`. A deleted library has no such fallback. A mapping whose file cannot be opened or read prints `UNVERIFIED which=map_file` with its range. The main executable with no descriptor is skipped, since step 2 has reported it.

The length is the mapping size, clamped to what the file holds past the mapping offset. If the file ends at or before the offset, the mapping is skipped and not counted. Otherwise `wtn` reads memory at `start + o` and the file at `offset + o`, in chunks of up to 64 KiB, counts each differing byte, keeps the position of the first, and feeds both sides to two SHA-256 states. `pread` is used throughout, so the shared descriptor keeps no position. Any difference emits `MEM_MISMATCH` with the count, the address of the first difference and both hashes.

If `mem` cannot be opened, or a read fails while the process lives, `wtn` prints `UNVERIFIED which=mem` once and compares nothing more. The four walks of the table still run, because they need only `maps`.

An anonymous mapping has no file to compare. If it is executable and its name is empty or begins `[`, `wtn` emits `ANON_EXEC`, except for `[vdso]`, `[vsyscall]` and `[vvar]`. Names that begin with neither `/` nor `[`, such as `anon_inode:`, match no check. `WX_MAP` applies to any mapping with `rwx`, file-backed or not. `LIB_DELETED` applies to an executable non-main file mapping with the suffix, which includes a non-main memfd. `LIB_ODD_PATH` applies to an executable non-main file mapping outside `/lib/`, `/lib64/`, `/usr/lib/`, `/usr/lib64/`, `/usr/local/lib/` and `/usr/libexec/`, and skips `/memfd:` names.

8.5. Parsing stat

`comm` is field 2 and may hold any bytes, so `wtn` searches for the last `)` in the file and starts at field 3 two bytes later. From there it splits on spaces. It keeps field 7, `tty_nr`, and fields 48 to 51: `arg_start`, `arg_end`, `env_start`, `env_end`. Too few fields, or no `)`, prints `UNVERIFIED which=stat`.

The kernel prints these four addresses as 0 for a reader without ptrace access, as `proc_pid_stat(5)` documents. Zero lies outside any stack, so a plain range test would report every such process. When all four are 0, `wtn` prints `UNVERIFIED which=stat_mm`, which counts as a failure to look and not as a finding. The range test, the stray scan and `LINEAGE` are then skipped.

8.6. The environment checks

The range test needs `maps` and a `[stack]` entry. Without the entry it prints `UNVERIFIED which=stack`. If `env_start` lies below the stack or `env_end` above it, the finding is `! ENV_RELOCATED`. The same test on the argument range gives `? ARGV_RELOCATED`. The levels differ because moved argument ranges occur on an idle machine: four processes had them in the scan below, and none had a moved environment.

`environ` is read whole, up to 128 KiB, and split on NUL. An entry beginning `LD_PRELOAD=`, `LD_LIBRARY_PATH=` or `LD_AUDIT=` emits `ENV_LOADER_VAR` with its value. This check runs even after `stat_mm`.

The byte scan needs an open `mem` and a parsed `stat`. It reads `[stack]` then `[heap]`, at most 64 MiB of each, in chunks of 64 KiB. Consecutive chunks overlap by 32 bytes, so a string cannot fall across a boundary unseen. The scan looks for the same three names followed by `=`, anywhere in the bytes. A hit inside the current `env_start` to `env_end` range is the live environment and is ignored. Any other hit emits `ENV_STRAY_LOADER_VAR` with its address. Reports stop at 8 per pid, across both regions. The check matches text, so it cannot tell an old environment from a script that mentions the variable.

8.7. The identity checks

`comm` is at most 15 visible bytes for a user process, so comparisons with it use the first 15 bytes of each side. `ID_COMM_EXE` compares `comm` with the basename of the link target, suffix removed. `ID_COMM_ARGV` compares `comm` with the basename of `argv[0]`. `ID_ARGV_EXE` compares the two basenames in full. An empty `comm`, an empty `argv[0]` or an unreadable link skips the checks that need it. Case 01 shows why the truncation matters: its 26 byte name produces no finding.

From `status`, `wtn` takes the real and effective uids from the `Uid:` line, `CapEff:`, `NSpid:` and `PPid:`. A non-zero effective uid with a non-zero `CapEff` emits `CAPS_UNPRIVILEGED`. The executable's mode is not read, so a setuid or setgid bit is neither required nor excluded. `fgetxattr` on the step 2 descriptor asks for `security.capability`, and the finding carries `filecaps=yes` or `filecaps=no`. `ENODATA` and `ENOTSUP` mean no; any other error prints `UNVERIFIED which=filecaps`. A real uid different from the effective uid gives the information line `IDS_SPLIT`. More than one value in `NSpid:` gives `NS_NESTED`, with the values joined by `/`. `PPid:` of 1, with `stat` parsed, gives `LINEAGE` with the first line of `cgroup` after its last `:`, and the `tty_nr` value from step 5.

8.8. Output

One finding is one line of fields separated by single spaces:

```
<pid> <level> <KIND> <key=value ...>
```

The pid is `-` for the checks tied to no process. There are four levels. `!` means two witnesses contradict each other. `?` means a disagreement that needs a person's judgement. `u` means a witness could not look. `.` is information, printed only with `-v`.

Strings in values are escaped as `\ooo`, three octal digits: every byte up to and including space, DEL and backslash. `comm` and argv are chosen by the process, and a newline in either would otherwise print a line that reads as a finding. Bytes above DEL pass unchanged. Addresses are hexadecimal with `0x`, and a range is two addresses joined by `-`.

With `-x`, a byte comparison is followed by a dump of both sides: the row of 16 bytes holding the first difference and the rows either side, then `^^` under each differing byte of that row. Memory rows are labelled with virtual addresses and file rows with offsets.

The exit status is 1 if any `!` or `?` line was printed, otherwise 3 if any `u` line was, otherwise 0. Usage errors, a failure to open `/proc` and a failure to write output exit with 2. `GONE` is level `.`, so a pid that does not exist prints nothing and exits with 0 without `-v`.

A failure to read is always printed, at level `u` and regardless of `-v`. A check that fails silently looks like a check that passed, and the exit status 3 keeps the two apart.

8.9. Finding kinds

The table lists every kind in `wtn.c` in source order.

kind	level	emitted when
`GONE`	`.`	a read found the process vanished
`UNVERIFIED`	`u`	a read was refused or failed; `which=` names `status`, `comm`, `cmdline`, `exe_link`, `exe`, `root`, `host`, `maps`, `maps_line`, `maps_cap`, `mem`, `map_file`, `stat`, `stat_mm`, `stack`, `environ`, `cgroup`, `filecaps` or `ld_so_preload`
`KTHREAD`	`.`	`status` has no `VmSize:`: a kernel thread or a zombie
`ID_COMM_EXE`	`?`	`comm` differs from the link basename in 15 bytes
`ID_COMM_ARGV`	`?`	`comm` differs from the `argv[0]` basename in 15 bytes
`ID_ARGV_EXE`	`?`	the `argv[0]` and link basenames differ
`EXE`	`.`	every readable process: link text, `comm`, `argv[0]`
`HASH`	`.`	a witness was hashed: `exe`, `root` or `host`
`EXE_DELETED`	`?`	link text ends `(deleted)`
`EXE_MEMFD`	`?`	link text begins `/memfd:`
`PATH_ROOT_MISMATCH`	`!`	ROOT hash differs from EXE
`PATH_ROOT_MISSING`	`?`	ROOT path absent, process alive
`PATH_HOST_MISSING`	`.`	HOST path absent
`PATH_HOST_MISMATCH`	`?`	HOST hash differs from EXE
`PATH_GONE`	`.`	deleted or memfd: no path to open
`MEM_MISMATCH`	`!`	a compared mapping differs from its file
`ANON_EXEC`	`?`	executable mapping, name empty or `[...]`, not `[vdso]`, `[vsyscall]`, `[vvar]`
`WX_MAP`	`?`	permissions begin `rwx`
`LIB_DELETED`	`?`	executable non-main file mapping ends `(deleted)`

kind	level	emitted when
LIB_ODD_PATH	?	executable non-main file mapping outside the six directories
MAPS	.	maps was read: mapping count and mappings compared
ENV_RELOCATED	!	environ range outside [stack]
ARGV_RELOCATED	?	argument range outside [stack]
ENV_LOADER_VAR	?	a loader variable is in environ
ENV_STRAY_LOADER_VAR	?	a loader variable name in [stack] or [heap] outside the environ range
CAPS_UNPRIVILEGED	?	non-zero euid, non-zero CapEff
LINEAGE	.	PPid: is 1 and stat was parsed
IDS_SPLIT	.	real uid differs from effective uid
NS_NESTED	.	NSpid: has more than one value
LDSO_PRELOAD	?	with -a, /etc/ld.so.preload is not empty

8.10. Linking, storage and the race

The default Makefile target links with -static. A statically linked program does not pass through ld.so, so the loader variables that wtn inspects in other processes have no effect on wtn, and the single file can be copied into a container or another root.

The program does not call malloc. Its storage is static, with two 64 KiB buffers and a table of 4096 mapping records in 17107776 bytes of bss, because the bounds are known and a comparison then has no allocation failure to handle. opendir and stdio do allocate inside libc.

Every read of /proc is racy: the process may change or exit between any two reads, so a finding describes a moment.

8.11. The testbed

The testbed has one small C program per case. Cases 00 and 01 are controls. A case prints its pid on the first line of output and waits. If it cannot run, it prints the reason and exits with 77, which run.sh records as a skip.

testbed/run.sh builds wtn and every case, starts each case with its environment, waits 0.3 seconds, runs ./wtn -x -v PID into results/, and kills the case. A case passes when every expected kind appears, no forbidden kind appears, and no ! kind appears that was not expected. A control also fails on any ? line. Cases 11 and 12 must match a pattern as well: via=exe and diff_bytes=1. Case 08 forbids ENV_LOADER_VAR, case 16 forbids PATH_ROOT_MISMATCH and case 18 forbids PATH_HOST_MISMATCH. The script then runs ./wtn -a on the idle machine and testbed/facts.sh.

It refuses to run without real root. id -u alone is not enough, because uid 0 in a user namespace passes that test and lacks what the cases need. The script also requires /proc/self/uid_map to read 0 0 4294967295, the identity map of the initial user namespace, and exits with 2 otherwise.

The run on 7 November 2026, with the information kinds EXE, HASH, MAPS, PATH_GONE and PATH_HOST_MISSING left out:

case	found	result
00-control	none	PASS
01-longname	none	PASS
02-argv_exec	ID_ARGV_EXE ID_COMM_ARGV	PASS
03-argv_overwrite	ID_ARGV_EXE ID_COMM_ARGV	PASS
04-prctl_name	ID_COMM_ARGV ID_COMM_EXE	PASS
05-deleted	EXE_DELETED	PASS
06-memfd	EXE_DELETED EXE_MEMFD ID_ARGV_EXE ID_COMM_ARGV	PASS
07-preload	ENV_LOADER_VAR LIB_ODD_PATH	PASS
08-preload_scrub	LIB_ODD_PATH	PASS
09-libpath	ENV_LOADER_VAR LIB_ODD_PATH	PASS
10-env_prctl	ENV_RELOCATED ENV_STRAY_LOADER_VAR LIB_ODD_PATH	PASS
11-exe_repoint	ANON_EXEC ID_ARGV_EXE ID_COMM_EXE MEM_MISMATCH	PASS
12-selfpatch	MEM_MISMATCH	PASS
13-anon_exec	ANON_EXEC WX_MAP	PASS
14-filecaps	CAPS_UNPRIVILEGED	PASS
15-setuid	IDS_SPLIT	PASS
16-mntns_pidns	NS_NESTED PATH_HOST_MISMATCH	PASS
17-orphan	LINEAGE	PASS
18-root_swap	PATH_ROOT_MISMATCH	PASS

19 pass, 0 fail, 0 skip

8.12. Reading the output

`MEM_MISMATCH` for case 12, a process that flipped one byte of its own text:

```
11150 ! MEM_MISMATCH map=0x560b1d8c0000-0x560b1d8c1000 off=0x1000 path=/home/v4n/work/witness/ ...
  mem  0000560b1d8c0240: f3 0f 1e fa e9 77 ff ff  ff 0f 1f 80 00 00 00 00  |.....w..........|
  mem  0000560b1d8c0250: 72 44 7f 01 c3 66 66 2e  0f 1f 84 00 00 00 00 00  |rD...ff.........|
  mem  0000560b1d8c0260: 55 48 89 fd 53 48 83 ec  08 e8 c2 fd ff ff 8b 18  |UH..SH..........|
  file 0000000000001240: f3 0f 1e fa e9 77 ff ff  ff 0f 1f 80 00 00 00 00  |.....w..........|
  file 0000000000001250: 8d 44 7f 01 c3 66 66 2e  0f 1f 84 00 00 00 00 00  |.D...ff.........|
  diff                   ^^
  file 0000000000001260: 55 48 89 fd 53 48 83 ec  08 e8 c2 fd ff ff 8b 18  |UH..SH..........|
```

The file is untouched, so EXE and both PATHs agree. `0x8d` became `0x72`. `PATH_HOST_MISMATCH` for case 16, around the first differing byte:

```
11240 ? PATH_HOST_MISMATCH path=/usr/bin/sleep exe_sha=54ac9dca0576f9104931709ad4441066dc23135 ...
  exe  0000000000000000: 7f 45 4c 46 02 01 01 03  00 00 00 00 00 00 00 00  |.ELF............|
  exe  0000000000000010: 02 00 3e 00 01 00 00 00  80 16 40 00 00 00 00 00  |..>.......@.....|
  host 0000000000000000: 7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
  diff                                     ^^
  host 0000000000000010: 03 00 3e 00 01 00 00 00  80 2a 00 00 00 00 00 00  |..>......*......|
```

A breakpoint gives the same line as case 12. I planted one with `PTRACE_POKETEXT`, and `wtn` printed `MEM_MISMATCH` with `diff_bytes=1`. Any write to compared text produces this finding. Uprobes, user-space live patching, kernel live patching and text relocations were not run. JIT compilers hold code in anonymous memory, which has no file to compare with and so is only reported. Wrappers and interpreters produce `ID_*` lines.

The scan of the idle machine gave 56 lines, with no `!` and no `u`. They are 25 `ID_*` lines, such as `python3.14` run as `python3` and `udevadm` run as `systemd-udevd`, and 16 `ENV_STRAY_LOADER_VAR` lines from two `run.sh` shells, a script that holds those names as text. The rest are 4 `ARGV_RELOCATED` lines from `(sd-pam)` and `systemd-userwork`, 4 `CAPS_UNPRIVILEGED` lines without file capabilities, 3 pairs of `ANON_EXEC` and `WX_MAP`, and 1 `PATH_ROOT_MISSING` for `rtkit-daemon`, the pattern of Section 6. I did not inspect the anonymous mappings or read that `root` link, and the two shells exited before I could look at them. I reviewed each line and found no contradiction among them.

Without root the witnesses narrow, and the tool says so. `/proc/1/stat` read zeros in the address fields as uid 1000, and `wtn` printed `u UNVERIFIED which=stat_mm`. `map_files` opened for real root only: the owner listed it and read the links with `CapEff` zero, which `proc_pid_map_files(5)` says should need a capability, and the open failed with `EPERM`, so `wtn` falls back to `via=root_path`. Under Yama, I measured `mem` denied to an unrelated reader of the same uid at scope 1, and to the parent too at scope 2, with `maps` and `environ` open throughout. Scope 3 was not run, because `ptrace(2)` says it cannot be changed once written.

9. Scope

The procedure takes the kernel's own file, memory and process code as given, and everything below it: hypervisor, SMM, management engine or platform security processor, microcode, silicon. A layer that can alter what the layer above observes cannot be audited by it. The witnesses in this paper are all reached through that kernel, so their agreement is agreement within what it reports. This is where the paper stops, and the stopping point is a choice.

10. Related work

Thompson's Turing lecture makes the argument one layer down: "You can't trust code that you did not totally create yourself." [8] Killian [1] and Plan 9 `proc(3)` [2] give the idea of process as file, and Plan 9′s `text` corresponds to `exe`. The talk by Kalvāns [5] is the starting point. The Linux manual pages [6] are the reference for each file, and Sections 4 to 8 record where they differ from what I saw. Volatility 3 has a Linux plugin that lists process memory ranges of interest from a memory image [9]. Comparing memory with disk is not a new idea, and I do not claim it is.

References

[1] Killian, T. J. Processes as files. In *Proc. USENIX Summer Conference* (Salt Lake City, June 1984), 203-207.

[2] proc(3). *Plan 9 Programmer's Manual*. https://9p.io/magic/man2html/3/proc

[3] procfs(5). *FreeBSD File Formats Manual*, as quoted on freebsd-questions, August 2025. https://lists.freebsd.org/archives/freebsd-questions/2025-August/006814.html

[4] OpenBSD 5.7 release notes, May 1, 2015. https://www.openbsd.org/57.html
[5] Kalvāns, A. What userland utilities won't tell you: uncovering hidden process state in procfs. SREcon26 EMEA (Dublin, Oct. 13, 2026), USENIX. https://www.usenix.org/conference/srecon26emea/presentation/kalvans
[6] Linux man-pages 6.19: `proc(5)` and its `proc_pid_*(5)` pages, `prctl(2)`, `PR_SET_MM(2const)` and related, `PR_SET_NAME(2const)`, `memfd_create(2)`, `capabilities(7)`, `namespaces(7)`, `ptrace(2)`, `ld.so(8)`.
[7] ps(1). procps-ng 4.0.7.
[8] Thompson, K. Reflections on trusting trust. *Commun. ACM 27*, 8 (Aug. 1984), 761-763.
[9] Volatility Foundation. Volatility 3 documentation, Linux plugins. https://volatility3.readthedocs.io/

Appendix: listings

The sources named in the text, then `wtn` output samples. In the body, lines wider than the page are cut and end in …; here they are whole.

testbed/03-argv_overwrite.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include "c.h"

int main(int ac, char **av)
{
        char *e = av[ac - 1] + strlen(av[ac - 1]);

        memset(av[0], 0, (size_t)(e - av[0]));
        strcpy(av[0], "[kworker/0:1]");
        sp(getpid());
        hold();
}
```

testbed/04-prctl_name.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include "c.h"

int main(void)
{
        if (prctl(PR_SET_NAME, "septicon", 0, 0, 0) < 0)
                sk("PR_SET_NAME");
        sp(getpid());
        hold();
}
```

testbed/libs.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include <string.h>

extern char **environ;

__attribute__((constructor)) static void scrub(void)
{
        char **e;

        for (e = environ; *e; e++)
                if (!strncmp(*e, "LD_PRELOAD=", 11))
                        memset(*e, 0, strlen(*e));
}
```

testbed/10-env_prctl.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include "c.h"

int main(void)
{
```

```
        char **e;
        size_t n = 1, l = 0;
        char *b;

        for (e = environ; *e; e++)
                n += strlen(*e) + 1;
        b = malloc(n);
        if (!b)
                sk("malloc");
        for (e = environ; *e; e++) {
                if (!strncmp(*e, "LD_", 3))
                        continue;
                strcpy(b + l, *e);
                l += strlen(*e) + 1;
        }
        if (prctl(PR_SET_MM, PR_SET_MM_ENV_START, (unsigned long)b, 0, 0) < 0)
                sk("PR_SET_MM_ENV_START");
        if (prctl(PR_SET_MM, PR_SET_MM_ENV_END, (unsigned long)(b + l), 0, 0) < 0)
                sk("PR_SET_MM_ENV_END");
        sp(getpid());
        hold();
}
```

testbed/11-exe_repoint.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include "c.h"

static void mv(unsigned long s, unsigned long e, const char *r)
{
        size_t z = e - s;
        int pr = (r[0] == 'r' ? PROT_READ : 0) | (r[1] == 'w' ? PROT_WRITE : 0) | (r[2] == 'x' ?
PROT_EXEC : 0);
        char *t = mmap(NULL, z, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

        if (t == MAP_FAILED)
                sk("mmap");
        memcpy(t, (void *)s, z);
        if (mprotect(t, z, pr) < 0 || mremap(t, z, z, MREMAP_MAYMOVE | MREMAP_FIXED, (void *)s) ==
MAP_FAILED)
                sk("mremap");
}

static void unmap(void)
{
        static unsigned long s[32], e[32];
        static char r[32][8];
        char l[4608], p[4096], x[4096];
        ssize_t n = readlink("/proc/self/exe", x, sizeof x - 1);
        FILE *f = fopen("/proc/self/maps", "r");
        int c = 0, i;

        if (n < 0 || !f)
                sk("maps");
        x[n] = 0;
        while (c < 32 && fgets(l, sizeof l, f)) {
                p[0] = 0;
                sscanf(l, "%lx-%lx %7s %*s %*s %*s %4095s", &s[c], &e[c], r[c], p);
                c += !strcmp(p, x);
        }
        fclose(f);
        for (i = 0; i < c; i++)
                mv(s[i], e[i], r[i]);
}
```

```
int main(void)
{
        int f = open("/bin/true", O_RDONLY);
        char *p;

        if (f < 0)
                sk("open /bin/true");
        if (prctl(PR_SET_MM, PR_SET_MM_EXE_FILE, (unsigned long)f, 0, 0) < 0) {
                if (errno != EBUSY)
                        sk("PR_SET_MM_EXE_FILE");
                unmap();
                if (prctl(PR_SET_MM, PR_SET_MM_EXE_FILE, (unsigned long)f, 0, 0) < 0)
                        sk("PR_SET_MM_EXE_FILE after unmap");
        }
        p = mmap(NULL, 4096, PROT_READ | PROT_WRITE, MAP_PRIVATE, f, 0x1000);
        if (p == MAP_FAILED)
                sk("mmap /bin/true");
        p[100] ^= 0xff;
        if (mprotect(p, 4096, PROT_READ | PROT_EXEC) < 0)
                sk("mprotect r-x");
        sp(getpid());
        hold();
}
```

testbed/12-selfpatch.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include "c.h"

__attribute__((noinline, used)) static int unu(int x)
{
        return x * 3 + 1;
}

int main(void)
{
        volatile unsigned char *p = (volatile unsigned char *)unu;
        char *g = (char *)((unsigned long)unu & ~4095UL);

        if (mprotect(g, 4096, PROT_READ | PROT_WRITE | PROT_EXEC) < 0)
                sk("mprotect rwx");
        *p ^= 0xff;
        if (mprotect(g, 4096, PROT_READ | PROT_EXEC) < 0)
                sk("mprotect r-x");
        sp(getpid());
        hold();
}
```

testbed/16-mntns_pidns.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include "c.h"

int main(int ac, char **av)
{
        const char *p = wt("sleep");
        pid_t c;

        if (ac < 2)
                sk("no payload");
```

```
        cpf(av[1], p, 0755);
        c = fork();
        if (c < 0)
                sk("fork");
        if (!c) {
                execlp("unshare", "unshare", "--mount", "--pid", "--fork", "sh", "-c",
                        "mount --bind \"$0\" /bin/sleep && exec /bin/sleep 1000", p, (char *)NULL);
                _exit(127);
        }
        wk(c);
        return 0;
}
```

testbed/18-root_swap.c

```
/*
 * Copyright (C) 2026 Ivan Gaydardzhiev
 * Licensed under the GPL-3.0-only
 */

#include "c.h"

int main(int ac, char **av)
{
        const char *p = wt("sleep");
        pid_t c;

        if (ac < 2)
                sk("no payload");
        cpf(av[1], p, 0755);
        c = fork();
        if (c < 0)
                sk("fork");
        if (!c) {
                execlp("unshare", "unshare", "--mount", "sh", "-c",
                        "/bin/sleep 1000 & p=$!;"
                        "until [ \"$(readlink /proc/$p/exe)\" = /usr/bin/sleep ]; do :; done;"
                        "mount --bind \"$0\" /bin/sleep; wait", p, (char *)NULL);
                _exit(127);
        }
        wk(c);
        return 0;
}
```

probe: an empty command line

```
/* probe: empties its own command line with PR_SET_MM_MAP, no capability */
#include "../testbed/c.h"
#include <linux/prctl.h>

int main(void)
{
        struct prctl_mm_map m;
        char b[1024], *q;
        unsigned long v[64];
        int i, f = open("/proc/self/stat", O_RDONLY);
        ssize_t n = read(f, b, sizeof b - 1);

        close(f);
        b[n > 0 ? n : 0] = 0;
        q = strrchr(b, ')') + 2;
        for (i = 3; i <= 52; i++) {
                char *e;

                if (*q >= 'A') {
                        v[i] = 0;
                        q += 2;
                        continue;
                }
```

```
                v[i] = strtoul(q, &e, 10);
                q = e + 1;
        }
        memset(&m, 0, sizeof m);
        m.start_code = v[26];
        m.end_code = v[27];
        m.start_stack = v[28];
        m.start_data = v[45];
        m.end_data = v[46];
        m.start_brk = v[47];
        m.brk = (unsigned long)sbrk(0);
        m.arg_start = v[48];
        m.arg_end = v[48];
        m.env_start = v[50];
        m.env_end = v[51];
        m.exe_fd = -1;
        if (prctl(PR_SET_MM, PR_SET_MM_MAP, &m, sizeof m, 0) < 0)
                sk("PR_SET_MM_MAP");
        prctl(PR_SET_NAME, "kworker/0:9", 0, 0, 0);
        sp(getpid());
        hold();
}
```

```
output of case 12
11150 . EXE path=/home/v4n/work/witness/testbed/bin/12-selfpatch comm=12-selfpatch argv0=./12-
selfpatch
11150 . HASH witness=exe sha=e4ee2765953e22543ca1528e6b157224934c10493f89da6e62235b44b22add19
size=16464
11150 . HASH witness=root sha=e4ee2765953e22543ca1528e6b157224934c10493f89da6e62235b44b22add19
size=16464
11150 . HASH witness=host sha=e4ee2765953e22543ca1528e6b157224934c10493f89da6e62235b44b22add19
size=16464
11150 ! MEM_MISMATCH map=0x560b1d8c0000-0x560b1d8c1000 off=0x1000 path=/home/v4n/work/witness/
testbed/bin/12-selfpatch via=exe diff_bytes=1 first=0x560b1d8c0250
mem_sha=7a59f69f081546f65a6dc754442f1e84647357c1153fef3da5e9825707c8de12
file_sha=6dbaaec7ee2c056127de98c685e05adf79ccd31750264bad624d20901d342348
  mem  0000560b1d8c0240: f3 0f 1e fa e9 77 ff ff  ff 0f 1f 80 00 00 00 00  |.....w..........|
  mem  0000560b1d8c0250: 72 44 7f 01 c3 66 66 2e  0f 1f 84 00 00 00 00 00  |rD...ff.........|
  mem  0000560b1d8c0260: 55 48 89 fd 53 48 83 ec  08 e8 c2 fd ff ff 8b 18  |UH..SH..........|
  file 0000000000001240: f3 0f 1e fa e9 77 ff ff  ff 0f 1f 80 00 00 00 00  |.....w..........|
  file 0000000000001250: 8d 44 7f 01 c3 66 66 2e  0f 1f 84 00 00 00 00 00  |.D...ff.........|
  diff                   ^^
  file 0000000000001260: 55 48 89 fd 53 48 83 ec  08 e8 c2 fd ff ff 8b 18  |UH..SH..........|
11150 . MAPS count=23 exec_checked=3
```

```
output of case 11
11129 ? ID_COMM_EXE comm=11-exe_repoint exe=/usr/bin/true
11129 ? ID_ARGV_EXE argv0=./11-exe_repoint exe=/usr/bin/true
11129 . EXE path=/usr/bin/true comm=11-exe_repoint argv0=./11-exe_repoint
11129 . HASH witness=exe sha=69745c15f1bbe866f4001a30b64081161a90dc8e669922e921296021ce55ebb3
size=43464
11129 . HASH witness=root sha=69745c15f1bbe866f4001a30b64081161a90dc8e669922e921296021ce55ebb3
size=43464
11129 . HASH witness=host sha=69745c15f1bbe866f4001a30b64081161a90dc8e669922e921296021ce55ebb3
size=43464
11129 ! MEM_MISMATCH map=0x7f56540c0000-0x7f56540c1000 off=0x1000 path=/usr/bin/true via=exe
diff_bytes=1 first=0x7f56540c0064
mem_sha=769f39d63e3efec5d87f2ab597a9dffbdc5bc707a8cf915602e6b6a3ef040e6f
file_sha=aadadf830401394d3cbdd8476035f335c5aa741df17a75f4516da9f601a21423
  mem  00007f56540c0050: c0 a0 00 00 00 00 00 00  05 00 00 00 37 00 00 00  |............7...|
  mem  00007f56540c0060: 00 00 00 00 ff 00 00 00  80 9e 00 00 00 00 00 00  |................|
  mem  00007f56540c0070: 07 00 00 00 01 00 00 00  00 00 00 00 00 00 00 00  |................|
  file 0000000000001050: c0 a0 00 00 00 00 00 00  05 00 00 00 37 00 00 00  |............7...|
  file 0000000000001060: 00 00 00 00 00 00 00 00  80 9e 00 00 00 00 00 00  |................|
  diff                                ^^
  file 0000000000001070: 07 00 00 00 01 00 00 00  00 00 00 00 00 00 00 00  |................|
```

```
11129 ? ANON_EXEC map=0x401000-0x496000 perms=r-xp name=-
11129 . MAPS count=12 exec_checked=1

output of case 16
11240 . EXE path=/usr/bin/sleep comm=sleep argv0=/bin/sleep
11240 . HASH witness=exe sha=54ac9dca0576f9104931709ad4441066dc23135a8c2470b864630eeb3f01f233
size=764176
11240 . HASH witness=root sha=54ac9dca0576f9104931709ad4441066dc23135a8c2470b864630eeb3f01f233
size=764176
11240 . HASH witness=host sha=83c0db85657e2cd1799b5e113317252d7865dd78157148d8757fab0ede52b1db
size=47560
11240 ? PATH_HOST_MISMATCH path=/usr/bin/sleep
exe_sha=54ac9dca0576f9104931709ad4441066dc23135a8c2470b864630eeb3f01f233
host_sha=83c0db85657e2cd1799b5e113317252d7865dd78157148d8757fab0ede52b1db
  exe  0000000000000000: 7f 45 4c 46 02 01 01 03  00 00 00 00 00 00 00 00  |.ELF............|
  exe  0000000000000010: 02 00 3e 00 01 00 00 00  80 16 40 00 00 00 00 00  |..>.......@.....|
  host 0000000000000000: 7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
  diff                                   ^^
  host 0000000000000010: 03 00 3e 00 01 00 00 00  80 2a 00 00 00 00 00 00  |..>......*......|
11240 . MAPS count=11 exec_checked=1
11240 . NS_NESTED nspid=11240/1

output of case 18
11289 . EXE path=/usr/bin/sleep comm=sleep argv0=/bin/sleep
11289 . HASH witness=exe sha=83c0db85657e2cd1799b5e113317252d7865dd78157148d8757fab0ede52b1db
size=47560
11289 . HASH witness=root sha=54ac9dca0576f9104931709ad4441066dc23135a8c2470b864630eeb3f01f233
size=764176
11289 ! PATH_ROOT_MISMATCH path=/usr/bin/sleep
exe_sha=83c0db85657e2cd1799b5e113317252d7865dd78157148d8757fab0ede52b1db
root_sha=54ac9dca0576f9104931709ad4441066dc23135a8c2470b864630eeb3f01f233 exe_size=47560
root_size=764176
  exe  0000000000000000: 7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
  exe  0000000000000010: 03 00 3e 00 01 00 00 00  80 2a 00 00 00 00 00 00  |..>......*......|
  root 0000000000000000: 7f 45 4c 46 02 01 01 03  00 00 00 00 00 00 00 00  |.ELF............|
  diff                                   ^^
  root 0000000000000010: 02 00 3e 00 01 00 00 00  80 16 40 00 00 00 00 00  |..>.......@.....|
11289 . HASH witness=host sha=83c0db85657e2cd1799b5e113317252d7865dd78157148d8757fab0ede52b1db
size=47560
11289 . MAPS count=26 exec_checked=3

output for the planted breakpoint
20089 ! MEM_MISMATCH map=0x558c08346000-0x558c08347000 off=0x1000 path=$B/p-brk via=exe
diff_bytes=1 first=0x5
  mem  0000558c083461c0: 00 00 01 5d c3 0f 1f 00  c3 0f 1f 80 00 00 00 00  |...]............|
  mem  0000558c083461d0: f3 0f 1e fa e9 77 ff ff  ff cc 44 7f 01 c3 48 8d  |.....w....D...H.|
  mem  0000558c083461e0: 64 24 d8 e8 e8 fe ff ff  85 c0 75 2f b9 00 00 00  |d$........u/....|
  file 00000000000011c0: 00 00 01 5d c3 0f 1f 00  c3 0f 1f 80 00 00 00 00  |...]............|
  file 00000000000011d0: f3 0f 1e fa e9 77 ff ff  ff 8d 44 7f 01 c3 48 8d  |.....w....D...H.|
  diff                                      ^^
  file 00000000000011e0: 64 24 d8 e8 e8 fe ff ff  85 c0 75 2f b9 00 00 00  |d$........u/....|

map_files and Yama measurements
target pid=11702 comm=sleep uid=0
target pid=11703 comm=sleep uid=1000
map_files reader=root open=ok
map_files reader=root_on_other_uid open=ok
map_files reader=owner open=denied
map_files reader=other_uid open=denied
scope=0 set=0
target pid=11726 comm=sleep uid=1000
reader=root: mem=ok maps=ok environ=ok
reader=same_uid_unrelated: mem=ok maps=ok environ=ok
reader=same_uid_parent: mem=ok maps=ok environ=ok
wtn_as_same_uid_unrelated:
scope=1 set=1
target pid=11742 comm=sleep uid=1000
reader=root: mem=ok maps=ok environ=ok
```

```
reader=same_uid_unrelated: mem=denied maps=ok environ=ok
reader=same_uid_parent: mem=ok maps=ok environ=ok
wtn_as_same_uid_unrelated: u UNVERIFIED which=mem;
scope=2 set=2
target pid=11758 comm=sleep uid=1000
reader=root: mem=ok maps=ok environ=ok
reader=same_uid_unrelated: mem=denied maps=ok environ=ok
reader=same_uid_parent: mem=denied maps=ok environ=ok
wtn_as_same_uid_unrelated: u UNVERIFIED which=mem;
restored ptrace_scope=0
```